

The two bugs are connected:

1. **Unlimited speed:** `self.speed += self.speed_increase_per_frame` runs on every frame forever. We can keep the same structure and add a maximum speed, e.g. `max_speed = 12`.
2. **Collision tunneling:** the obstacle is moved by the full speed before collision is checked. At high speed, it can jump from one side of the player to the other between frames. We can fix this without changing the overall structure by checking the **movement path** using the obstacle's previous position and its current position.

Here is the corrected `engine.py`. The class structure, methods, and game flow remain the same.

```
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)

        self.speed = 6
        self.max_speed = 12
        self.speed_increase_per_frame = 0.003

        self.spawn_interval = 70 # frames between obstacle spawns
        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
```

```
if event.type == pygame.KEYDOWN and event.key in (
    pygame.K_SPACE,
    pygame.K_UP,
    pygame.K_w,
):
    self.player.jump()

def handle_input(self):
    # Reserved for continuously-held-key input; this runner only
    # needs an edge-triggered jump, handled in handle_event.
    pass

def update(self):
    if self.game_over:
        return

    # Increase speed, but never allow it to exceed max_speed.
    self.speed = min(
        self.speed + self.speed_increase_per_frame,
        self.max_speed
    )

    self.player.update()

    self._spawn_timer += 1
    if self._spawn_timer >= self.spawn_interval:
        self._spawn_timer = 0
        self.obstacles.append(
            Obstacle(self.width, self.ground_y, self.speed)
        )

    for obstacle in self.obstacles:
        # Save the obstacle's position before movement.
        previous_x = obstacle.x
```

```
obstacle.move()
obstacle.speed = self.speed

# Collision with the obstacle at its new position.
if obstacle.rect().colliderect(self.player.rect()):
    self.game_over = True
    return

# Check whether the obstacle crossed the player's horizontal
# hitbox during this frame.
player_rect = self.player.rect()

previous_rect = pygame.Rect(
    previous_x,
    obstacle.y,
    obstacle.width,
    obstacle.height,
)

# The obstacle's swept horizontal area from its previous
# position to its current position.
left = min(previous_rect.left, obstacle.rect().left)
right = max(previous_rect.right, obstacle.rect().right)

swept_rect = pygame.Rect(
    left,
    obstacle.y,
    right - left,
    obstacle.height,
)

if swept_rect.colliderect(player_rect):
    self.game_over = True
    return
```

```

for obstacle in self.obstacles:
    if not obstacle.scored and obstacle.x + obstacle.width < self.player.x:
        obstacle.scored = True
        self.score += 1

self.obstacles = [o for o in self.obstacles if not o.off_screen()]

self.distance += self.speed

def render(self, screen):
    pygame.draw.line(
        screen,
        BROWN,
        (0, self.ground_y),
        (self.width, self.ground_y),
        4,
    )

    pygame.draw.rect(screen, WHITE, self.player.rect())

    for obstacle in self.obstacles:
        pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        (0, 0, 0),
    )
    screen.blit(score_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        print("Game over! Final score:", self.score)
        self._game_over_logged = True

```

What changed

Speed cap:

</> Python



▶ Run

and:

```
self.speed = min(  
    self.speed + self.speed_increase_per_frame,  
    self.max_speed
```

</> Python



▶ Run

So the speed increases normally until 12 , then stays at 12 .

Collision tunneling fix:

Before moving:

</> Python



▶ Run

After moving, we create a rectangle covering the obstacle's **entire horizontal path during that frame**:

```
left = min(previous_rect.left, obstacle_rect().left)
```

</> Python



▶ Run

Then:

```
if swept_rect.colliderect(player_rect):  
    self.game_over = True
```

</> Python



▶ Run

So even if an obstacle moves from $x = 100 \rightarrow x = 50$ in one frame and completely jumps across the player's hitbox, the swept rectangle covers 50-100, and the collision is detected.

This keeps your existing `GameEngine` structure and does not require changing `Player`, `Obstacle`, or the rest of the project.



Okay this is working. Now that collision is reliable, I want to implement task 2 and 3.

Right now, dying just logs a message in the terminal. Can we update `engine.py` so that:

1. When `game_over` is `True`, it draws an in-game graphical overlay instead of just printing to the console. Let's display a clear "GAME OVER" title and show the player's final score.
2. Add a replay option on that screen where the player can choose a starting difficulty—like pressing 1 for Easy, 2 for Medium, or 3 for Hard—with adjusted speeds and spawn rates.

Show more ▾

Yes. We can keep the existing structure and preserve both fixes. The main changes are:

- `render()` now draws a **GAME OVER** overlay with score and controls.
- `handle_event()` handles **1/2/3 difficulty selection**, `R` / `Space` `retry`, and `Q` / `Esc` `quit`.
- `reset_game()` resets the player, score, timers, obstacles, distance, and speed.
- Easy/Medium/Hard change the **starting speed** and **spawn interval**.
- The **speed cap** and **swept collision detection** remain intact.

Replace your current `engine.py` with this:

```
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)
```

```
# Speed settings
self.speed = 6
self.max_speed = 12
self.speed_increase_per_frame = 0.003

# Default difficulty
self.spawn_interval = 70

self._spawn_timer = 0
self.obstacles = []

self.distance = 0
self.score = 0

self.font = pygame.font.SysFont("Arial", 30)
self.game_over_font = pygame.font.SysFont("Arial", 60, bold=True)
self.final_score_font = pygame.font.SysFont("Arial", 36, bold=True)
self.control_font = pygame.font.SysFont("Arial", 25)

self.game_over = False
self.difficulty = "Medium"

def handle_event(self, event):
    # Normal gameplay controls
    if not self.game_over:
        if event.type == pygame.KEYDOWN and event.key in (
            pygame.K_SPACE,
            pygame.K_UP,
            pygame.K_w,
        ):
            self.player.jump()

    # Game-over controls
    else:
        if event.type == pygame.KEYDOWN:
```

```
# Select difficulty and restart
if event.key == pygame.K_1:
    self.reset_game("Easy")

elif event.key == pygame.K_2:
    self.reset_game("Medium")

elif event.key == pygame.K_3:
    self.reset_game("Hard")

# Quick retry using the current difficulty
elif event.key in (pygame.K_r, pygame.K_SPACE):
    self.reset_game(self.difficulty)

# Quit the game
elif event.key in (pygame.K_q, pygame.K_ESCAPE):
    pygame.quit()
    raise SystemExit

def handle_input(self):
    # Reserved for continuously-held-key input.
    pass

def reset_game(self, difficulty="Medium"):
    """
    Reset the game so the player can start again without
    restarting the entire program.
    """

    # Difficulty settings
    difficulty_settings = {
        "Easy": {
            "speed": 4,
            "spawn_interval": 90,
```

```
    },
    "Medium": {
        "speed": 6,
        "spawn_interval": 70,
    },
    "Hard": {
        "speed": 8,
        "spawn_interval": 50,
    },
}

settings = difficulty_settings[difficulty]

# Reset player
self.player = Player(80, self.ground_y)

# Reset difficulty-dependent values
self.difficulty = difficulty
self.speed = settings["speed"]
self.spawn_interval = settings["spawn_interval"]

# Reset game state
self._spawn_timer = 0
self.obstacles = []
self.distance = 0
self.score = 0
self.game_over = False

# Allow game-over message to be printed again
self._game_over_logged = False

def update(self):
    if self.game_over:
        return
```

```
# Increase speed, but never allow it to exceed max_speed.
```

```
self.speed = min(  
    self.speed + self.speed_increase_per_frame,  
    self.max_speed,  
)
```

```
self.player.update()
```

```
self._spawn_timer += 1
```

```
if self._spawn_timer >= self.spawn_interval:  
    self._spawn_timer = 0
```

```
    self.obstacles.append(  
        Obstacle(  
            self.width,  
            self.ground_y,  
            self.speed,  
        )  
    )
```

```
for obstacle in self.obstacles:
```

```
    # Save the obstacle's position before movement.  
    previous_x = obstacle.x
```

```
    obstacle.move()  
    obstacle.speed = self.speed
```

```
    # Normal collision check at the new position.  
    if obstacle.rect().colliderect(self.player.rect()):  
        self.game_over = True  
        return
```

```
    # -----
```

```
# SWEPT COLLISION DETECTION
# -----
# Check whether the obstacle crossed the player's hitbox
# during this frame, even if it moved completely past it.

player_rect = self.player.rect()

previous_rect = pygame.Rect(
    previous_x,
    obstacle.y,
    obstacle.width,
    obstacle.height,
)

left = min(
    previous_rect.left,
    obstacle.rect().left,
)

right = max(
    previous_rect.right,
    obstacle.rect().right,
)

swept_rect = pygame.Rect(
    left,
    obstacle.y,
    right - left,
    obstacle.height,
)

if swept_rect.colliderect(player_rect):
    self.game_over = True
    return
```

```
# Update score
for obstacle in self.obstacles:
    if (
        not obstacle.scored
        and obstacle.x + obstacle.width < self.player.x
    ):
        obstacle.scored = True
        self.score += 1

# Remove obstacles that have gone off screen
self.obstacles = [
    obstacle
    for obstacle in self.obstacles
    if not obstacle.off_screen()
]

self.distance += self.speed

def render(self, screen):
    # -----
    # NORMAL GAME RENDERING
    # -----

    pygame.draw.line(
        screen,
        BROWN,
        (0, self.ground_y),
        (self.width, self.ground_y),
        4,
    )

    pygame.draw.rect(
        screen,
        WHITE,
        self.player.rect(),
```

```
)

for obstacle in self.obstacles:
    pygame.draw.rect(
        screen,
        DARK_GREEN,
        obstacle.rect(),
    )

# Score
score_text = self.font.render(
    f"Score: {self.score}",
    True,
    (0, 0, 0),
)

screen.blit(
    score_text,
    (10, 10),
)

# -----
# GAME OVER OVERLAY
# -----

if self.game_over:

    # Semi-transparent dark overlay
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA,
    )

    overlay.fill((0, 0, 0, 170))
```

```
screen.blit(
    overlay,
    (0, 0),
)

# GAME OVER title
game_over_text = self.game_over_font.render(
    "GAME OVER",
    True,
    WHITE,
)

game_over_rect = game_over_text.get_rect(
    center=(self.width // 2, self.height // 2 - 100)
)

screen.blit(
    game_over_text,
    game_over_rect,
)

# Final score
final_score_text = self.final_score_font.render(
    f"Final Score: {self.score}",
    True,
    WHITE,
)

final_score_rect = final_score_text.get_rect(
    center=(self.width // 2, self.height // 2 - 35)
)

screen.blit(
    final_score_text,
    final_score_rect,
```

```
)

# Difficulty selection
difficulty_text = self.control_font.render(
    "1 - Easy    2 - Medium    3 - Hard",
    True,
    WHITE,
)

difficulty_rect = difficulty_text.get_rect(
    center=(self.width // 2, self.height // 2 + 25)
)

screen.blit(
    difficulty_text,
    difficulty_rect,
)

# Retry / quit controls
controls_text = self.control_font.render(
    "R / SPACE - Retry    Q / ESC - Quit",
    True,
    WHITE,
)

controls_rect = controls_text.get_rect(
    center=(self.width // 2, self.height // 2 + 70)
)

screen.blit(
    controls_text,
    controls_rect,
)

# Print game-over message only once
```

```
if not getattr(self, "_game_over_logged", False):
    print(
        "Game over! Final score:",
        self.score,
    )
    self._game_over_logged = True
```

How the controls work

Key	Action
1	Restart on Easy
2	Restart on Medium
3	Restart on Hard
R	Retry with current difficulty
Space	Retry with current difficulty
Q	Quit
Esc	Quit

The starting difficulties are:

Difficulty	Starting Speed	Spawn Interval
Easy	4	90 frames
Medium	6	70 frames

Difficulty	Starting Speed	Spawn Interval
Hard	8	50 frames

The speed still gradually increases during gameplay, but **never exceeds** 12 , regardless of difficulty.

One important detail: I kept the swept collision exactly in the update flow from the previous fix. The new game-over/replay functionality doesn't interfere with it.



Today 6:24 PM

Yup its working. Now for Task 4 we need sound feedback for jumping, scoring, and game over. I do not want to download external .wav files. Can we use pygame.mixer to generate simplesynthetic tones programmatically so the repository remains self-contained?

Please make sure audio failures fail gracefully without crashing if a sound device isn't available.

Yes. We can keep the project completely self-contained by generating short tones in memory and passing them to `pygame.mixer.Sound` . No .wav files are needed.

The important part is that **mixer initialization and sound creation are wrapped in** `try/except` , so the game continues normally if audio isn't supported.

Below is the updated `engine.py` , keeping your **speed cap, swept collision, difficulty selection, replay/reset, and game-over overlay** intact.

```
import math
import array
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)

        # Speed settings
        self.speed = 6
        self.max_speed = 12
        self.speed_increase_per_frame = 0.003

        # Default difficulty
        self.spawn_interval = 70

        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
```

```
self.score = 0

self.font = pygame.font.SysFont("Arial", 30)
self.game_over_font = pygame.font.SysFont("Arial", 60, bold=True)
self.final_score_font = pygame.font.SysFont("Arial", 36, bold=True)
self.control_font = pygame.font.SysFont("Arial", 25)

self.game_over = False
self.difficulty = "Medium"

# Initialize sound effects.
self._init_audio()

# -----
# AUDIO
# -----

def _init_audio(self):
    """
    Create simple sound effects programmatically.

    No external audio files are required. If the mixer or audio
    device is unavailable, the game simply runs without sound.
    """

    self.audio_enabled = False
    self.jump_sound = None
    self.score_sound = None
    self.game_over_sound = None

    try:
        # Initialize the mixer if it is not already initialized.
        if not pygame.mixer.get_init():
            pygame.mixer.init()
```

```
self.jump_sound = self._create_tone(
    frequency=600,
    duration=0.10,
    volume=0.35,
)

self.score_sound = self._create_tone(
    frequency=900,
    duration=0.08,
    volume=0.35,
)

self.game_over_sound = self._create_tone(
    frequency=180,
    duration=0.30,
    volume=0.45,
)

self.audio_enabled = True

except (pygame.error, OSError):
    # Audio is optional. Do not crash if it is unavailable.
    self.audio_enabled = False

def _create_tone(self, frequency, duration, volume):
    """
    Generate a sine-wave tone in memory and return it as a
    pygame.mixer.Sound object.
    """

    sample_rate = 44100
    sample_count = int(sample_rate * duration)

    samples = array.array("h")
```

```
    amplitude = int(32767 * volume)

    for i in range(sample_count):
        sample = int(
            amplitude
            * math.sin(
                2 * math.pi * frequency * i / sample_rate
            )
        )

        samples.append(sample)

    return pygame.mixer.Sound(
        buffer=samples.tobytes()
    )

def _play_sound(self, sound):
    """
    Play a sound safely. Any audio failure is ignored so that
    sound problems never stop the game.
    """

    if not self.audio_enabled or sound is None:
        return

    try:
        sound.play()
    except pygame.error:
        pass

# -----
# INPUT
# -----

def handle_event(self, event):
```

```
# Normal gameplay controls
if not self.game_over:

    if event.type == pygame.KEYDOWN and event.key in (
        pygame.K_SPACE,
        pygame.K_UP,
        pygame.K_w,
    ):
        self.player.jump()

        # Jump sound
        self._play_sound(self.jump_sound)

# Game-over controls
else:

    if event.type == pygame.KEYDOWN:

        # Select difficulty and restart
        if event.key == pygame.K_1:
            self.reset_game("Easy")

        elif event.key == pygame.K_2:
            self.reset_game("Medium")

        elif event.key == pygame.K_3:
            self.reset_game("Hard")

        # Retry using current difficulty
        elif event.key in (
            pygame.K_r,
            pygame.K_SPACE,
        ):
            self.reset_game(self.difficulty)
```

```
        # Quit
        elif event.key in (
            pygame.K_q,
            pygame.K_ESCAPE,
        ):
            pygame.quit()
            raise SystemExit

    def handle_input(self):
        # Reserved for continuously-held-key input.
        pass

    # -----
    # GAME RESET
    # -----

    def reset_game(self, difficulty="Medium"):
        """
        Reset the game so the player can start again without
        restarting the entire program.
        """

        difficulty_settings = {
            "Easy": {
                "speed": 4,
                "spawn_interval": 90,
            },
            "Medium": {
                "speed": 6,
                "spawn_interval": 70,
            },
            "Hard": {
                "speed": 8,
                "spawn_interval": 50,
            },
        },
```

```
}

settings = difficulty_settings[difficulty]

# Reset player
self.player = Player(80, self.ground_y)

# Reset difficulty
self.difficulty = difficulty
self.speed = settings["speed"]
self.spawn_interval = settings["spawn_interval"]

# Reset game state
self._spawn_timer = 0
self.obstacles = []
self.distance = 0
self.score = 0
self.game_over = False

self._game_over_logged = False

# -----
# GAME UPDATE
# -----

def update(self):
    if self.game_over:
        return

    # Speed cap
    self.speed = min(
        self.speed + self.speed_increase_per_frame,
        self.max_speed,
    )
```

```
self.player.update()

self._spawn_timer += 1

if self._spawn_timer >= self.spawn_interval:
    self._spawn_timer = 0

    self.obstacles.append(
        Obstacle(
            self.width,
            self.ground_y,
            self.speed,
        )
    )

for obstacle in self.obstacles:

    # Save position before movement
    previous_x = obstacle.x

    obstacle.move()
    obstacle.speed = self.speed

    # Normal collision check
    if obstacle.rect().collidect(self.player.rect()):
        self.game_over = True

        # Game-over sound
        self._play_sound(self.game_over_sound)

    return

# -----
# SWEEP COLLISION DETECTION
# -----
```

```
player_rect = self.player.rect()

previous_rect = pygame.Rect(
    previous_x,
    obstacle.y,
    obstacle.width,
    obstacle.height,
)

left = min(
    previous_rect.left,
    obstacle.rect().left,
)

right = max(
    previous_rect.right,
    obstacle.rect().right,
)

swept_rect = pygame.Rect(
    left,
    obstacle.y,
    right - left,
    obstacle.height,
)

if swept_rect.colliderect(player_rect):
    self.game_over = True

    # Game-over sound
    self._play_sound(self.game_over_sound)

    return
```

```
# -----  
# SCORING  
# -----  
  
for obstacle in self.obstacles:  
  
    if (  
        not obstacle.scored  
        and obstacle.x + obstacle.width < self.player.x  
    ):  
        obstacle.scored = True  
        self.score += 1  
  
        # Score sound  
        self._play_sound(self.score_sound)  
  
# Remove off-screen obstacles  
self.obstacles = [  
    obstacle  
    for obstacle in self.obstacles  
    if not obstacle.off_screen()  
]  
  
self.distance += self.speed  
  
# -----  
# RENDER  
# -----  
  
def render(self, screen):  
  
    # Ground  
    pygame.draw.line(  
        screen,  
        BROWN,
```

```
        (0, self.ground_y),
        (self.width, self.ground_y),
        4,
    )

    # Player
    pygame.draw.rect(
        screen,
        WHITE,
        self.player.rect(),
    )

    # Obstacles
    for obstacle in self.obstacles:
        pygame.draw.rect(
            screen,
            DARK_GREEN,
            obstacle.rect(),
        )

    # Score
    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        (0, 0, 0),
    )

    screen.blit(
        score_text,
        (10, 10),
    )

    # -----
    # GAME OVER OVERLAY
    # -----
```

```
if self.game_over:

    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA,
    )

    overlay.fill((0, 0, 0, 170))

    screen.blit(
        overlay,
        (0, 0),
    )

    # GAME OVER title
    game_over_text = self.game_over_font.render(
        "GAME OVER",
        True,
        WHITE,
    )

    game_over_rect = game_over_text.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 100,
        )
    )

    screen.blit(
        game_over_text,
        game_over_rect,
    )

    # Final score
```

```
final_score_text = self.final_score_font.render(  
    f"Final Score: {self.score}",  
    True,  
    WHITE,  
)  
  
final_score_rect = final_score_text.get_rect(  
    center=(  
        self.width // 2,  
        self.height // 2 - 35,  
    )  
)  
  
screen.blit(  
    final_score_text,  
    final_score_rect,  
)  
  
# Difficulty selection  
difficulty_text = self.control_font.render(  
    "1 - Easy    2 - Medium    3 - Hard",  
    True,  
    WHITE,  
)  
  
difficulty_rect = difficulty_text.get_rect(  
    center=(  
        self.width // 2,  
        self.height // 2 + 25,  
    )  
)  
  
screen.blit(  
    difficulty_text,  
    difficulty_rect,
```

```
)

# Retry / quit controls
controls_text = self.control_font.render(
    "R / SPACE - Retry      Q / ESC - Quit",
    True,
    WHITE,
)

controls_rect = controls_text.get_rect(
    center=(
        self.width // 2,
        self.height // 2 + 70,
    )
)

screen.blit(
    controls_text,
    controls_rect,
)

# Console message only once
if not getattr(
    self,
    "_game_over_logged",
    False,
):
    print(
        "Game over! Final score:",
        self.score,
    )

    self._game_over_logged = True
```

What was added

1. Jump sound

Every successful jump now calls:

</> Python



▶ Run

2. Score sound

When an obstacle is successfully passed:

</> Python



▶ Run

```
self.score += 1  
self._play_sound(self.score_sound)
```

3. Game-over sound

Both collision paths trigger the same game-over sound, so the sound works whether the collision is detected normally or through the swept collision check.

4. No external files

The tones are generated using:

</> Python



▶ Run

```
math.sin(...)
```

and converted directly into a `pygame.mixer.Sound`. So your repository doesn't need a `sounds/` folder or any `.wav` files.

5. Graceful audio failure

If the machine has no working audio device, this:

</> Python



▶ Run

```
try:  
    pygame.mixer.init()
```

fails safely, `audio_enabled` remains `False`, and the game continues silently.

So the audio is essentially an **optional feature**—gameplay, collision, scoring, replay, etc. don't depend on it.



ChatGPT can make mistakes. Check important info.